

Trường ĐH Công Thương TP. HCM Khoa Công nghệ Thông tin Bộ môn Kỹ thuật phần mềm Công nghệ .NET	BÀI 6. MÔ HÌNH MVVM (tt)	
--	-------------------------------------	--

A. MỤC TIÊU

– Trình bày được vai trò của mô hình MVVM trong ứng dụng WPF và sự khác biệt giữa xử lý Event truyền thống và Command trong MVVM.

– Triển khai và sử dụng thành thạo Command (ICommand, RelayCommand) để xử lý các thao tác Thêm – Sửa – Xóa mà không sử dụng code-behind.

– Xây dựng được ứng dụng WPF theo mô hình MVVM

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SINH VIÊN

STT	Thiết bị - Phần mềm – Thư viện	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	
2	Microsoft Visual Studio 2013 trở lên	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Mô hình MVVM

1.2. Liên kết dữ liệu trong MVVM

1.3. Event và Command

Trong WPF, sự tương tác giữa người dùng và giao diện được xử lý chủ yếu thông qua Event (Sự kiện) và Command (Lệnh). Hai cơ chế này cho phép ứng dụng phản hồi các thao tác của người dùng như nhấp chuột, nhập dữ liệu và lựa chọn đối tượng.

1.3.1. Sự kiện Event

Event (Sự kiện) là cơ chế cho phép chương trình tiếp nhận và xử lý các hành động do người dùng thực hiện, chẳng hạn như:

- Click chuột

- Gõ phím
- Thay đổi nội dung
- Load cửa sổ

và thường được xử lý trong code-behind (.xaml.cs).

Đặc điểm của Event:

- Phù hợp ứng dụng nhỏ
- Gắn chặt View và xử lý logic
- Không phù hợp với MVVM quy mô lớn

Một số event thường gặp trong WPF:

Điều khiển	Event	Ý nghĩa
Button	Click	Nhấn nút
TextBox	TextChanged	Nội dung thay đổi
Window	Loaded	Cửa sổ được tải
ListBox	SelectionChanged	Thay đổi mục chọn
TextBox	GotFocus	Nhận focus

1.3.2. Command (Lệnh) trong WPF

Command là cơ chế thay thế Event trong MVVM, cho phép View gọi hành động thông qua binding.

Ưu điểm của Command:

- Không cần code-behind
- Dễ quản lý trạng thái Enable / Disable
- Phù hợp kiểm thử và mở rộng ứng dụng

WPF cung cấp Interface chuẩn:

```
public interface ICommand
{
    bool CanExecute(object parameter);
    void Execute(object parameter);
    event EventHandler CanExecuteChanged;
}
```

Trong đó:

- CanExecute: xác định command có được phép thực hiện hay không
- Execute: xử lý hành động
- CanExecuteChanged: thông báo UI cập nhật trạng thái.

1.3.3. RelayCommand

RelayCommand là lớp thực thi ICommand phổ biến trong MVVM:

- Không cần tạo nhiều class command riêng biệt
- Truyền trực tiếp hàm xử lý và điều kiện thực thi

RelayCommand thường dùng để:

- Điều khiển trạng thái nút Thêm / Sửa / Xóa
- Quản lý luồng xử lý Add – Edit – Save – Cancel
- Tách hoàn toàn xử lý khỏi View

1.4. Datagrid

Datagrid: Là một control trong WPF dùng để hiển thị dữ liệu dạng bảng (table), có hàng (Row) và cột (Column) rõ ràng. Hỗ trợ “Thêm / sửa / xóa dữ liệu”, chọn dòng, Sắp xếp (Sort).

Kết hợp với MVVM:

- ItemsSource: ObservableCollection
- SelectedItem: đối tượng đang chọn
- Tự động cập nhật khi dữ liệu thay đổi

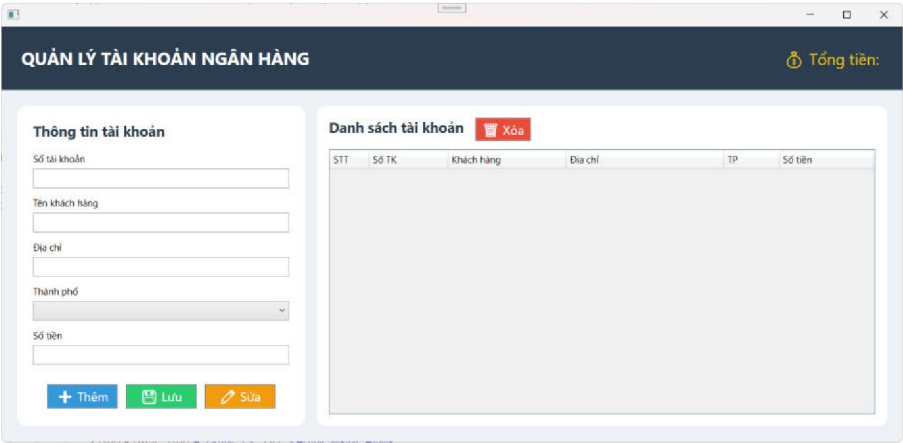
Cú pháp (xaml) thường dùng:

```
<DataGrid ItemsSource = "{Binding TenCollection}"
          SelectedItem = "{Binding ItemDangChon}"
          AutoGenerateColumns = "False"
          CanUserAddRows = "False" IsReadOnly = "False">
  <DataGrid.Columns>
    <!--Các cột-->
  </DataGrid.Columns>
</DataGrid>
```

2. Bài tập thực hành trên lớp

Bài tập có hướng dẫn

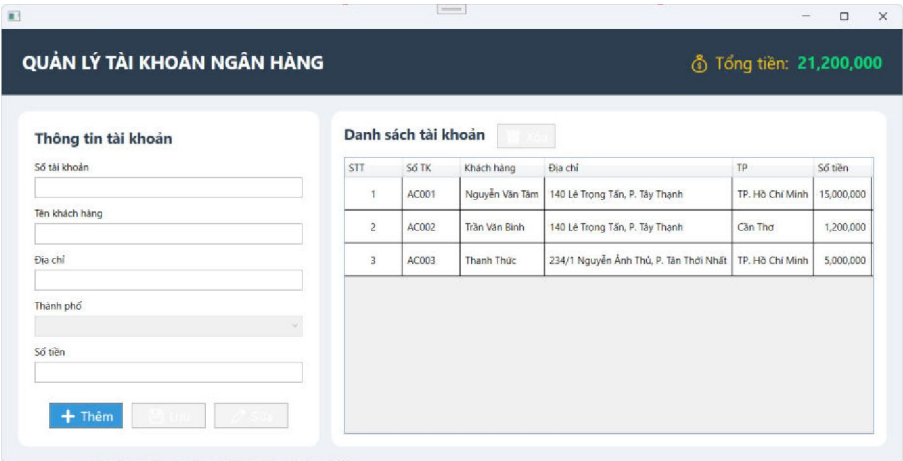
Bài 1. Thiết kế chương trình Quản lý thông tin tài khoản ngân hàng (WPF – MVVM).



Yêu cầu:

1. Khi Form được tải lên:

- DataGridView hiển thị ít nhất 01 dòng dữ liệu mẫu
- Dữ liệu được nạp từ ViewModel
- Tổng tiền được tính tự động
- Enable nút “Thêm”
- Disable các nút: “Lưu”, “Sửa”, “Xóa”



2. Trạng thái Thêm/ Hủy:

Khi nhấn nút Thêm:

- Chuyển nội dung nút thành “Hủy”
- Xóa trắng toàn bộ các TextBox / ComboBox
- Con trỏ tự động focus vào ô Số tài khoản
- Nút Lưu được Enable

Khi nhấn nút Hủy: hủy thao tác thêm, giao diện trở về trạng thái ban đầu

3. Chức năng Lưu dữ liệu:

- Chỉ được Enable khi đang ở chế độ Thêm hoặc Sửa
- Khi lưu:
 - **Thêm mới** hoặc **cập nhật** dữ liệu vào danh sách
 - DataGridView tự động cập nhật thông qua binding
 - Tự động cập nhật STT
 - Cập nhật lại Tổng tiền

4. Chọn và hiển thị dữ liệu: Khi người dùng chọn một dòng trong DataGridView thì thông tin tài khoản được hiển thị lên các TextBox / ComboBox.

5. Trạng thái Sửa/ Hủy:

- Nút Sửa chỉ Enable khi đã chọn một dòng trong DataGridView
- Khi nhấn nút Sửa:
 - Chuyển sang chế độ chỉnh sửa
 - Nút Sửa đổi text thành “Hủy”
- Khi nhấn nút Hủy: hủy thao tác sửa, giao diện trở về trạng thái ban đầu

6. Xóa dữ liệu:

- Nút Xóa chỉ Enable khi có dòng được chọn
- Xác nhận và Xóa tài khoản đang chọn khỏi danh sách
- DataGridView cập nhật tự động

- Cập nhật lại STT và Tổng tiền

7. Thực hiện Validate (kiểm tra ràng buộc dữ liệu):

- Không được để trống: Số tài khoản, Tên khách hàng
- Số tài khoản không được trùng
- Số tiền không được âm
- Dữ liệu không hợp lệ thì không cho lưu.

Hướng dẫn:

Bước 1. Chuẩn bị Models

Tạo class AccountModel trong thư mục Models

```
public class AccountModel: INotifyPropertyChanged
{
    private int _stt;
    private string _accountNumber;
    private string _customerName;
    private string _address;
    private string _city;
    private decimal _balance;

    public int STT
    {
        get { return _stt; }
        set
        {
            if (_stt != value)
            {
                _stt = value;
                OnPropertyChanged(nameof(STT));
            }
        }
    }
}
```

```
public string AccountNumber
{
    get
    {
        return _accountNumber;
    }
    set
    {
        if (_accountNumber != value)
        {
            _accountNumber = value;
            OnPropertyChanged(nameof(AccountNumber));
        }
    }
}

public string CustomerName
{
    get
    {
        return _customerName;
    }
    set
    {
        if (_customerName != value)
        {
            _customerName = value;
            OnPropertyChanged(nameof(CustomerName));
        }
    }
}
```

```
public string Address
{
    get
    {
        return _address;
    }
    set
    {
        if (_address != value)
        {
            _address = value;
            OnPropertyChanged(nameof(Address));
        }
    }
}
```

```
public string City
{
    get
    {
        return _city;
    }
    set
    {
        if (_city != value)
        {
            _city = value;
            OnPropertyChanged(nameof(City));
        }
    }
}
```

```

public decimal Balance
{
    get
    {
        return _balance;
    }
    set
    {
        if (_balance != value)
        {
            _balance = value;
            OnPropertyChanged(nameof(Balance));
        }
    }
}

public event PropertyChangedEventHandler PropertyChanged;
protected void OnPropertyChanged(string propertyName)
{
    PropertyChanged?.Invoke(this, new
        PropertyChangedEventArgs(propertyName));
}
}

```

Bước 2. Tạo ViewModels

1. Tạo class BaseViewModel: `INotifyPropertyChanged`
2. Tạo class RelayCommand thực thi ICommand

```

public class RelayCommand: ICommand
{
    private readonly Action<object> _execute;
    private readonly Predicate<object> _canExecute;
}

```

```

public RelayCommand(Action<object> execute,
                    Predicate<object> canExecute = null)
{
    _execute = execute ?? throw new
        ArgumentException(nameof(execute));
    _canExecute = canExecute;
}

public bool CanExecute(object parameter)
{
    return _canExecute == null || _canExecute(parameter);
}

public void Execute(object parameter)
{
    _execute(parameter);
}

public event EventHandler CanExecuteChanged
{
    add { CommandManager.RequerySuggested += value; }
    remove { CommandManager.RequerySuggested -= value; }
}

public void RaiseCanExecuteChanged()
{
    CommandManager.InvalidateRequerySuggested();
}
}

```

3. Tạo class **AccountViewModel** kế thừa **BaseViewModel**

```
public class AccountViewModel : BaseViewModel
{
    // =====DANH SÁCH & ĐỐI TƯỢNG=====
    public ObservableCollection<AccountModel> Accounts { get; set; }
    public ObservableCollection<string> Cities { get; set; }

    private AccountModel _selectedAccount;
    public AccountModel SelectedAccount
    {
        get => _selectedAccount;
        set
        {
            _selectedAccount = value;
            OnPropertyChanged();
            EditCommand.RaiseCanExecuteChanged();
            DeleteCommand.RaiseCanExecuteChanged();
        }
    }

    //Trạng thái nghiệp vụ (Add / Edit)
    private bool _isAdding;
    public bool IsAdding
    {
        get => _isAdding;
        set {
            _isAdding = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(AddButtonText));
            OnPropertyChanged(nameof(IsInputEnabled));
            SaveCommand.RaiseCanExecuteChanged();
        }
    }
}
```

```

private bool _isEditing;
public bool IsEditing
{
    get => _isEditing;
    set {
        _isEditing = value;
        OnPropertyChanged();
        OnPropertyChanged(nameof(EditButtonText));
        OnPropertyChanged(nameof(IsInputEnabled));
        SaveCommand.RaiseCanExecuteChanged();
    }
}

public bool IsInputEnabled => IsAdding || IsEditing;
public string AddButtonText => IsAdding ? "🗑️ Hủy" : "➕ Thêm";
public string EditButtonText => IsEditing ? "🗑️ Hủy Sửa" : "✎ Sửa";

public decimal TotalBalance
{
    get => Accounts?.Sum(a => a.Balance) ?? 0;
}

//Command
public RelayCommand AddCommand { get; }
public RelayCommand EditCommand { get; }
public RelayCommand SaveCommand { get; }
public RelayCommand DeleteCommand { get; }

```

```
public AccountViewModel()  
{  
    Accounts = new ObservableCollection<AccountModel>  
    {  
        new AccountModel  
        {  
            STT = 1,  
            AccountNumber = "AC001",  
            CustomerName = "Nguyễn Văn Tâm",  
            Address = "140 Lê Trọng Tấn, P. Tây Thạnh",  
            City = "TP. Hồ Chí Minh",  
            Balance = 15000000  
        },  
        new AccountModel  
        {  
            STT = 2,  
            AccountNumber = "AC002",  
            CustomerName = "Trần Văn Bình",  
            Address = "140 Lê Trọng Tấn, P. Tây Thạnh",  
            City = "Cần Thơ",  
            Balance = 1200000  
        },  
        new AccountModel  
        {  
            STT = 3,  
            AccountNumber = "AC003",  
            CustomerName = "Thanh Thức",  
            Address = "234/1 Nguyễn Ảnh Thủ, P. Tân Thới Nhất",  
            City = "TP. Hồ Chí Minh",  
            Balance = 5000000  
        }  
    };  
};
```

```
Cities = new ObservableCollection<string>
{
    "TP. Hồ Chí Minh",
    "Hà Nội",
    "Đà Nẵng",
    "Cần Thơ",
    "Huế"
};

// Bước 2: Khi thêm / xóa tài khoản
Accounts.CollectionChanged += (s, e) =>
{
    OnPropertyChanged(nameof(TotalBalance));
};

AddCommand = new RelayCommand(
    _ => AddOrCancel(),
    _ => !IsEditing
);

EditCommand = new RelayCommand(
    _ => EditOrCancel(),
    _ => SelectedAccount != null && !IsAdding
);

SaveCommand = new RelayCommand(
    _ => Save(),
    _ => IsAdding || IsEditing
);

DeleteCommand = new RelayCommand(
    _ => Delete(),
    _ => SelectedAccount != null && !IsAdding
);
```

```

    foreach (var acc in Accounts)
        acc.PropertyChanged += Account_PropertyChanged;
}

private void Account_PropertyChanged(object sender,
    System.ComponentModel.PropertyChangedEventArgs e)
{
    if (e.PropertyName == nameof(AccountModel.Balance))
    {
        OnPropertyChanged(nameof(TotalBalance));
    }
}

//Thêm / Hủy
private void AddOrCancel()
{
    if (IsAdding) {
        ResetState();
        return;
    }
    SelectedAccount = new AccountModel();
    IsAdding = true;
}

//Sửa / Hủy sửa
private void EditOrCancel()
{
    if (IsEditing) {
        ResetState();
        return;
    }
    IsEditing = true;
}

```

```

//Luu(Add + Edit + Validate)
private void Save()
{
    if (!Validate())
        return;
    if (IsAdding)
    {
        SelectedAccount.STT = Accounts.Count + 1;
        Accounts.Add(SelectedAccount);
        SelectedAccount.PropertyChanged +=
Account_PropertyChanged;
    }
    ResetState();
}

//Xóa
private void Delete()
{
    if (SelectedAccount == null)
        return;

    if (MessageBox.Show("Bạn có chắc muốn xóa?",
        "Xác nhận",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question) != DialogResult.Yes)
        return;
    SelectedAccount.PropertyChanged -= Account_PropertyChanged;
    Accounts.Remove(SelectedAccount);
    for (int i = 0; i < Accounts.Count; i++)
        Accounts[i].STT = i + 1;
    SelectedAccount = null;
}

```

```

//Validate dữ liệu
private bool Validate()
{
    if
(string.IsNullOrEmpty(SelectedAccount.AccountNumber))
    {
        MessageBox.Show("Số tài khoản không được để trống");
        return false;
    }
    if (string.IsNullOrEmpty(SelectedAccount.CustomerName))
    {
        MessageBox.Show("Tên khách hàng không được để trống");
        return false;
    }
    if (SelectedAccount.Balance < 0)
    {
        MessageBox.Show("Số tiền không hợp lệ");
        return false;
    }
    bool isDuplicate = Accounts.Any(a =>
        a.AccountNumber == SelectedAccount.AccountNumber &&
        a != SelectedAccount);
    if (isDuplicate)
    {
        MessageBox.Show("Số tài khoản đã tồn tại");
        return false;
    }
    return true;
}

```

```

//Reset trạng thái
private void ResetState() {
    IsAdding = false;
    IsEditing = false;
    SelectedAccount = null;
}
}

```

Bước 3. Binding trong XAML

```

<TextBlock Text="Số tài khoản"/>
<TextBox x:Name="txt_SoTK"
    Text="{Binding SelectedAccount.AccountNumber}"/>

```

```

<TextBlock Text="Tên khách hàng"/>
<TextBox Text="{Binding SelectedAccount.CustomerName}"/>

```

```

<TextBlock Text="Địa chỉ"/>
<TextBox Text="{Binding SelectedAccount.Address}"/>

```

```

<TextBlock Text="Thành phố"/>
<ComboBox ItemsSource="{Binding Cities}"
    SelectedItem="{Binding SelectedAccount.City,
    Mode=TwoWay}"
    IsEnabled="{Binding IsInputEnabled}"/>

```

```

<TextBlock Text="Số tiền"/>
<TextBox Text="{Binding SelectedAccount.Balance,
    StringFormat=N0}"/>

```

```

<Button x:Name="btn_Them" Content="{Binding AddButtonText}"
    Command="{Binding AddCommand}"
    Click="btn_Them_Click"/>

```

Tương tự cho các button còn lại. Và Datagrid chứa danh sách tài khoản:

```
<DataGrid Grid.Row="1"
    ItemsSource="{Binding Accounts}"
    SelectedItem="{Binding SelectedAccount}"
    AutoGenerateColumns="False" CanUserAddRows="False"
    IsReadOnly="True" RowHeight="40"
    ColumnWidth="SizeToCells" MinColumnWidth="70"
    HeadersVisibility="Column">
    <DataGrid.Columns>
        <DataGridTextColumn Header="STT" Binding="{Binding STT}"
            Width="50"/>
        <DataGridTextColumn Header="Số TK"
            Binding="{Binding AccountNumber}" Width="100"/>
        <DataGridTextColumn Header="Khách hàng" Width="150"
            Binding="{Binding CustomerName}" />
        <DataGridTextColumn Header="Địa chỉ"
            Binding="{Binding Address}" Width="200"/>
        <DataGridTextColumn Header="TP" Binding="{Binding City}"
            Width="70"/>
        <DataGridTextColumn Header="Số tiền" Width="120"
            Binding="{Binding Balance, StringFormat=N0}" />
    </DataGrid.Columns>
</DataGrid>
```

Bài tập sinh viên tự làm

Bài 2. Thiết kế chương trình quản lý hồ sơ nhân viên sử dụng WPF theo mô hình MVVM với giao diện như hình

Quản lý nhân viên theo phòng ban (MVVM)

Chọn phòng ban ở bên trái để xem danh sách nhân viên bên phải. CRUD thao tác bằng Command.

Tổng nhân viên: 3

Phòng ban

- Giám đốc
- Quản trị
- Kế hoạch
- Kế toán

Quản lý phòng ban

+ Thêm

Xóa phòng ban

Danh sách nhân viên (DataGrid lọc theo phòng ban đang chọn) **Số lượng nhân viên: 2**

Mã Nhân viên	Họ tên	Địa chỉ	Phòng ban
E001	Nguyễn Văn Bình	TP. Hồ Chí Minh	Giám đốc
E002	Trần Thị Hương Dương	Tây Ninh	Giám đốc

Mã Nhân viên Phòng ban Giám đốc

Họ tên Địa chỉ

Thao tác

- + Thêm
- Sửa
- Lưu
- Xóa

Yêu cầu:

1. Khi Form được tải lên

- TreeView hiển thị sẵn danh sách phòng ban tối thiểu gồm: Giám đốc, Quản trị, Kế hoạch.
- Tự động chọn một phòng ban mặc định (ví dụ: phòng đầu tiên).
- DataGrid hiển thị danh sách nhân viên đúng theo phòng ban đang chọn.
- Thống kê:
 - Tổng nhân viên = tổng số nhân viên của tất cả phòng ban.
 - Số lượng nhân viên = số nhân viên thuộc phòng ban chọn.
- Trạng thái nút:
 - Enable: Thêm (nhân viên)
 - Disable: Lưu, Sửa, Xóa (nếu chưa chọn nhân viên)

2. Quản lý phòng ban (TreeView + ComboBox đồng bộ)

- **Thêm phòng ban**
 - Nhập tên phòng ban mới sẽ hiển thị nút “Thêm”.
 - Không cho phép tên phòng ban trống.

- Không cho phép trùng tên phòng ban.
- Sau khi thêm:
 - TreeView cập nhật có phòng ban mới
 - ComboBox phòng ban cập nhật đồng bộ

▪ **Xóa phòng ban**

- Chỉ cho phép xóa khi đang chọn một phòng ban.
- Hiện thị hộp thoại xác nhận.
- Sau khi xóa: TreeView và ComboBox cập nhật đồng bộ
- Phòng ban được chọn lại hợp lý (ví dụ: phòng kế tiếp)

3. Quản lý nhân viên

- Khi chọn một phòng ban trên TreeView:
 - ComboBox “Phòng ban” đồng bộ theo phòng ban đang chọn.
 - DataGrid tự động lọc và hiển thị danh sách nhân viên thuộc phòng ban đó.
 - Cập nhật số lượng nhân viên theo phòng ban.
- Khi chọn một dòng nhân viên trong DataGrid:
 - Thông tin nhân viên được đổ lên các TextBox/ComboBox để xem.
- Trạng thái các nút Thêm, Sửa, Lưu, Xóa (nhân viên) thực hiện tương tự bài 1.
- Validate dữ liệu nhân viên
 - Không được để trống: Mã nhân viên, Họ tên
 - Mã nhân viên không được trùng trong cùng một phòng ban
 - Địa chỉ không bắt buộc, nhưng nếu nhập thì tối thiểu X ký tự (tùy bạn quy định)
 - Dữ liệu không hợp lệ thì không cho lưu và hiển thị thông báo.

Hướng dẫn:

Bước 1: Chuẩn bị Models

1. **Tạo class Employee** trong thư mục Models: dùng để lưu và hiển thị thông tin nhân viên và binding dữ liệu

```
public class Employee: InotifyPropertyChanged {
    private string _employeeId;
    private string _fullName;
    private string _address;
    public string EmployeeId {
        get => _employeeId;
        set { _employeeId = value; OnPropertyChanged(); }
    }
    public string FullName {
        get => _fullName;
        set { _fullName = value; OnPropertyChanged(); }
    }
    public string Address {
        get => _address;
        set { _address = value; OnPropertyChanged(); }
    }
    public Employee() { }
    public Employee(string id, string name, string address) {
        EmployeeId = id;
        FullName = name;
        Address = address;
    }
    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string
                                   propertyName = null)
    {
        PropertyChanged?.Invoke(this, new
                                   PropertyChangedEventArgs(propertyName));
    }
}
```

2. **Tạo class Department** trong thư mục Models: Đại diện cho phòng ban, Quản lý danh sách nhân viên thuộc phòng ban. Khi thêm / xóa nhân viên, giao diện tự động cập nhật.

```
public class Department: INotifyPropertyChanged
{
    private string _name;
    public string Name {
        get => _name;
        set { _name = value; OnPropertyChanged(); }
    }
    public ObservableCollection<Employee> Employees { get; set; } =
        new ObservableCollection<Employee>();

    public Department() { }
    public Department(string name) => Name = name;
    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName]
        string propertyName = null)
    {
        PropertyChanged?.Invoke(this,
            new PropertyChangedEventArgs(propertyName));
    }
}
```

Bước 2: Tạo ViewModel

1. Tạo **BaseViewModel** (INotifyPropertyChanged)

2. Tạo **RelayCommand**

(giống bài 1)

3. **Tạo class PBNVViewModel** trong thư mục ViewModels kế thừa BaseViewModel

 **Thuộc tính**

```
public ObservableCollection<Department> Departments { get; set; }
    = new ObservableCollection<Department>();
```

```

private Department _selectedDepartment;
public Department SelectedDepartment {
    get => _selectedDepartment;
    set {
        if (_selectedDepartment == value) return;
        _selectedDepartment = value;
        OnPropertyChanged();
        if (!IsAdding && !IsEditing)
            SelectedDepartmentInForm = _selectedDepartment;
        SelectedEmployee = null;
        OnPropertyChanged(nameof(EmployeeCountInSelectedDepartment));
        OnPropertyChanged(nameof(TotalEmployeeCount));

        EditCommand.RaiseCanExecuteChanged();
        DeleteCommand.RaiseCanExecuteChanged();
        AddCommand.RaiseCanExecuteChanged();
        RemoveDepartmentCommand.RaiseCanExecuteChanged();
    }
}

private Employee _selectedEmployee;
public Employee SelectedEmployee {
    get => _selectedEmployee;
    set {
        _selectedEmployee = value;
        OnPropertyChanged();
        if (_selectedEmployee != null && !IsAdding && !IsEditing) {
            EmployeeIdInput = _selectedEmployee.EmployeeId;
            FullNameInput = _selectedEmployee.FullName;
            AddressInput = _selectedEmployee.Address;
            SelectedDepartmentInForm = SelectedDepartment;
        }
        EditCommand.RaiseCanExecuteChanged();
        DeleteCommand.RaiseCanExecuteChanged();
    }
}

```

```

private string _employeeIdInput;
public string EmployeeIdInput {
    get => _employeeIdInput;
    set {
        _employeeIdInput = value;
        OnPropertyChanged();
        SaveCommand.RaiseCanExecuteChanged();
    }
}

private string _fullNameInput;
public string FullNameInput {
    get => _fullNameInput;
    set {
        _fullNameInput = value;
        OnPropertyChanged();
        SaveCommand.RaiseCanExecuteChanged(); }
}

private string _addressInput;
public string AddressInput {
    get => _addressInput;
    set {
        _addressInput = value;
        OnPropertyChanged();
        SaveCommand.RaiseCanExecuteChanged(); }
}

private string _newDepartmentName;
public string NewDepartmentName {
    get => _newDepartmentName;
    set {
        _newDepartmentName = value;
        OnPropertyChanged();
        AddDepartmentCommand.RaiseCanExecuteChanged(); }
}

```

```

private string _errorMessage;
public string ErrorMessage
{
    get => _errorMessage;
    set { _errorMessage = value; OnPropertyChanged(); }
}
// ===== State =====
private bool _isAdding;
public bool IsAdding {
    get => _isAdding;
    set
    {
        _isAdding = value;
        OnPropertyChanged();
        OnPropertyChanged(nameof(AddButtonText));
        OnPropertyChanged(nameof(IsInputEnabled));
        OnPropertyChanged(nameof(IsEmployeeIdEnabled));
        SaveCommand.RaiseCanExecuteChanged();
    }
}
private bool _isEditing;
public bool IsEditing {
    get => _isEditing;
    set
    {
        _isEditing = value;
        OnPropertyChanged();
        OnPropertyChanged(nameof(EditButtonText));
        OnPropertyChanged(nameof(IsInputEnabled));
        OnPropertyChanged(nameof(IsEmployeeIdEnabled));
        SaveCommand.RaiseCanExecuteChanged();
    }
}

```

```

public bool IsInputEnabled => IsAdding || IsEditing;
public bool IsEmployeeIdEnabled => IsAdding;
public string AddButtonText => IsAdding ? "↩ Hủy Thêm" : "➕ Thêm";
public string EditButtonText => IsEditing ? "↩ Hủy Sửa" : "➡ Sửa";

// ===== Statistics =====
public int EmployeeCountInSelectedDepartment =>
    SelectedDepartment?.Employees.Count ?? 0;
public int TotalEmployeeCount =>
    Departments.Sum(d => d.Employees.Count);

```

Commands

```

public RelayCommand AddCommand { get; }
public RelayCommand EditCommand { get; }
public RelayCommand SaveCommand { get; }
public RelayCommand DeleteCommand { get; }

public RelayCommand AddDepartmentCommand { get; }
public RelayCommand RemoveDepartmentCommand { get; }

```

Phương thức tạo dữ liệu ban đầu

```

private void CreatedData()
{
    var d1 = new Department("Giám đốc");
    d1.Employees.Add(new Employee("E001", "Nguyễn Văn Bình",
                                   "TP. Hồ Chí Minh"));
    d1.Employees.Add(new Employee("E002", "Trần Thị Dương",
                                   "Tây Ninh"));

    var d2 = new Department("Quản trị hệ thống");
    var d3 = new Department("Kế hoạch tài chính");
    Departments.Add(d1);
    Departments.Add(d2);
    Departments.Add(d3);
}

```

Phương thức khởi tạo

```
public PBNVViewModel()
{
    CreatedData();

    AddCommand = new RelayCommand(_ => AddOrCancel(),
                                   _ => SelectedDepartment != null && !IsEditing);
    EditCommand = new RelayCommand(_ => EditOrCancel(),
                                    _ => SelectedEmployee != null && !IsAdding);
    SaveCommand = new RelayCommand(_ => Save(),
                                    _ => (IsAdding || IsEditing));
    DeleteCommand = new RelayCommand(_ => Delete(),
                                     _ => SelectedEmployee != null && !IsAdding && !IsEditing);

    AddDepartmentCommand = new RelayCommand(
        _ => AddDepartment(),
        _ => !string.IsNullOrEmpty(NewDepartmentName));

    RemoveDepartmentCommand = new RelayCommand(
        _ => RemoveDepartment(),
        _ => SelectedDepartment != null && !IsAdding && !IsEditing);

    //default
    SelectedDepartment = Departments.FirstOrDefault();
    SelectedDepartmentInForm = SelectedDepartment;
}
```

Thêm/ Xóa phòng ban

```
private void AddDepartment()
{
    ErrorMessage = "";
    var name = (NewDepartmentName ?? "").Trim();
    if (string.IsNullOrEmpty(name)) return;
```

```

bool dup = Departments.Any(d => d.Name.Equals(name,
                                     StringComparison.OrdinalIgnoreCase));

if (dup)
{
    MessageBox.Show("Phòng ban này đã tồn tại!", "Cảnh báo",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

var dep = new Department(name);
Departments.Add(dep);
NewDepartmentName = "";
}

```

```

private void RemoveDepartment()
{
    if (SelectedDepartment == null) return;

    var result = MessageBox.Show("Bạn muốn xóa phòng ban này?",
                                "Xác nhận",
                                MessageBoxButton.YesNo,
                                MessageBoxImage.Question);

    if (result != MessageBoxResult.Yes) return;
    var depToRemove = SelectedDepartment;
    var index = Departments.IndexOf(depToRemove);
    Departments.Remove(depToRemove);
    SelectedDepartment = Departments.Count == 0 ? null :
        Departments[Math.Min(index, Departments.Count - 1)];
    SelectedDepartmentInForm = SelectedDepartment;
}

```

Thực hiện tương tự cho chức năng Thêm/Xóa/Sửa/Lưu nhân viên.

Bước 4: Binding trong XAML

1. **DataContext**: nguồn dữ liệu mặc định cho toàn bộ UI trong Window, giúp UI tự động lấy dữ liệu từ ViewModel

```
<Window
    ...
    xmlns:vm="clr-namespace: _00_HovaTen_Tuan6.ViewModels"
    ...>

<Window.DataContext>
    <vm:PBNVViewModel/>
</Window.DataContext>
```

2. **TreeView** phòng ban:

```
<TreeView x:Name="tvDepartments"
    ItemsSource="{Binding Departments}"
    BorderThickness="0"
    Background="Transparent"
    SelectedItemChanged="tvDepartments_SelectedItemChanged">

    <TreeView.ItemTemplate>
        <DataTemplate>
            <TextBlock Text="{Binding Name}" Padding="4"/>
        </DataTemplate>
    </TreeView.ItemTemplate>
</TreeView>
```

3. **ComboBox** phòng ban khi thêm nhân viên:

```
<ComboBox Grid.Row="0" Grid.Column="3" Padding="5"
    ItemsSource="{Binding Departments}"
    DisplayMemberPath="Name"
    SelectedItem="{Binding SelectedDepartment, Mode=TwoWay}"
    IsEnabled="{Binding IsInputEnabled}"
    Grid.ColumnSpan="2" />
```

4. TextBox thêm nhân viên, phòng ban:

```
<TextBox Text="{Binding EmployeeIdInput}"
        IsEnabled="{Binding IsEmployeeIdEnabled}"/>
<TextBox Text="{Binding FullNameInput}"
        IsEnabled="{Binding IsInputEnabled}"/>
<TextBox Text="{Binding AddressInput}"
        IsEnabled="{Binding IsInputEnabled}"/>
<TextBox Text="{Binding NewDepartmentName}"/>
```

5. Button Commands:

```
<Button Content=" + Thêm"
        Command="{Binding AddDepartmentCommand}"/>
<Button Content="{Binding AddButtonText}"
        Command="{Binding AddCommand}"/>
...
```

Bước 5: Xử lý code-behind

```
private void tvDepartments_SelectedItemChanged_1(object sender,
        RoutedPropertyChangedEventArgs<object> e)
{
    if (DataContext is
        _00_HovaTen_Tuan6.ViewModels.PBNVViewModel vm)
    {
        if (e.NewValue is
            _00_HovaTen_Tuan6.Models.Department dep)
            vm.SelectedDepartment = dep;
    }
}
```

3. Bài tập về nhà:

Bài 3. Thiết kế chương trình quản lý lớp học sử dụng WPF (.NET) theo mô hình MVVM với giao diện như hình

The screenshot displays a WPF application window titled "QUẢN LÝ LỚP HỌC (MVVM)". The interface is divided into several sections:

- Top Bar:** Contains the title "QUẢN LÝ LỚP HỌC (MVVM)" and a subtitle "Lọc theo lớp • Tìm theo Mã/ Tên sinh viên • Tính điểm trung bình".
- Left Panel:** Titled "Danh sách lớp", it contains a "Quản lý lớp" section with a "Thêm / Xóa một lớp." label, a text input field, a "+ Thêm" button, and a "Xóa lớp" button.
- Main Content Area:**
 - Search Section:** Includes a "Nhập thông tin tìm kiếm:" label, a text input field, a "Tìm" button, and a "Hiển thị tất cả" button.
 - Table Section:** Titled "Danh sách sinh viên", it features a table with columns: "Mã sinh viên", "Họ và Tên", "Địa chỉ", "Điểm 1", "Điểm 2", "Điểm 3", and "Điểm TB". The table body is currently empty.
 - Form Section:** Contains input fields for "Mã Sinh viên", "Họ và Tên", "Địa chỉ", and "Điểm" (with sub-fields for "Điểm 1", "Điểm 2", and "Điểm 3"). It also includes a "Lớp" dropdown menu and a "Thao tác" section with buttons for "+ Thêm", "Sửa", "Xóa", and "Lưu".

Yêu cầu:

1. Khi Form được tải lên
 - Hiển thị sẵn một số lớp mẫu.
 - Mỗi lớp có một số sinh viên mẫu.
 - Tự động chọn lớp đầu tiên.
 - DataGridView hiển thị danh sách sinh viên của lớp đó.
2. Khi người dùng chọn một lớp:
 - DataGridView cập nhật danh sách sinh viên tương ứng.
 - Cập nhật thống kê số lượng sinh viên của lớp.
3. Chức năng Thêm/ Xóa lớp thực hiện tương tự bài trước
4. Chức năng Thêm/Xóa/Sửa sinh viên được mô tả tương tự bài trước.
5. Tìm kiếm sinh viên
 - Nhập từ khóa vào ô tìm kiếm.
 - Nhấn nút Tìm:
 - Lọc danh sách sinh viên của lớp đang chọn.

- So khớp theo: Mã sinh viên, hoặc Họ tên (Contains)
- Nhân nút Hiển thị tất cả:
 - Xóa bộ lọc
 - Hiển thị toàn bộ sinh viên của lớp.

6. Tính điểm trung bình

Điểm trung bình được tính theo công thức:

$$(\text{Điểm 1} + \text{Điểm 2} + \text{Điểm 3}) / 3$$

- Tính tự động.
- Hiển thị trong DataGrid.
- Là thuộc tính chỉ đọc.
- Làm tròn 2 chữ số thập phân.

7. Validate dữ liệu

- Tên lớp không được để trống khi thêm
- Mã sinh viên không được để trống.
- Họ tên không được để trống.
- Điểm phải nằm trong khoảng 0 – 10.
- Mã sinh viên không được trùng.

Nếu sai:

- Không cho lưu.
- Hiển thị thông báo lỗi.